Module 1 – Overview of IT Industry

Q1.What is a Program?

A **program** is a set of **instructions** that tells a computer **what to do** and **how to do it.**

THEORY EXERCISE: Explain in your own words what a program is and how it functions.

A **program** is a set of instructions that tells a computer how to perform a task. It works by being translated into machine language and then executed step by step by the computer to take input, process data, and produce output.

During execution, a program can:

- **Accept input** (from the user, files, or other programs)
- **Process data** using logic and calculations
- **Produce output** such as results on the screen or stored data

Q2.What is Programming?

Programming is the process of writing instructions that tell a computer how to perform tasks or solve problems.

THEORY EXERCISE: What are the key steps involved in the programming process?

The key steps in the programming process are **problem analysis, designing a solution (algorithm), writing the code, testing and debugging, and maintaining the program**.

Q3. Types of Programming Languages

The main types of programming languages are **machine language, assembly language, and high-level languages**.

THEORY EXERCISE: What are the main differences between high-level and low-level programminglanguages?

High-level languages are **easy to read and write** and are closer to human language, while low-level languages are **closer to machine code**, harder to write, and give more control over hardware.

Q4. World Wide Web & How Internet Works

The **World Wide Web (WWW)** is a system of interconnected web pages accessed through the Internet using a web browser. The **Internet** works by connecting computers worldwide and transferring data using standard protocols like **TCP/IP**.

LAB EXERCISE: Research and create a diagram of how data is transmitted from a client to a server over the internet.

```
[ Client (Browser) ]

        |

        | 1. Request (HTTP/HTTPS)

        v

[ Router / ISP ]

        |

        | 2. Data split into packets (TCP/IP)

        v

[ Internet ]

 (Multiple Networks)

        |

        | 3. Packets routed to destination

        v

[ Server (Web Server) ]
```

|

| 4. Response sent back

v

[Client (Browser)]

THEORY EXERCISE: Describe the roles of the client and server in web communication.

In web communication, the **client** sends requests for information or services (such as opening a web page), and the **server** receives these requests, processes them, and sends the requested data or response back to the client.

Q5.Network Layers on Client and Server

On both the client and server, network communication is organized into layers: **Application, Transport, Network, Data Link, and Physical**, where each layer has a specific role in sending and receiving data.

THEORY EXERCISE: Explain the function of the TCP/IP model and its layers.

The **TCP/IP model** explains how data is transmitted over the Internet using four layers: **Application, Transport, Internet, and Network Access**, where each layer has a specific role in sending, routing, and receiving data.

Q6. Client and Servers

Client: A client is a device or program that requests services or data.
Server: A server is a device or program that receives requests and provides services or data to clients.

THEORY EXERCISE: Explain Client Server Communication

Client–server communication is a process where a client sends a request to a server, and the server processes the request and sends a response back to the client.

Q7. Types of Internet Connections

Common **types of Internet connections** include **dial-up, DSL, cable, fiber-optic, satellite, and mobile (4G/5G).**

THEORY EXERCISE: How does broadband differ from fiber-optic internet?

Broadband uses copper cables and offers moderate speeds, while fiber-optic uses light through glass fibers to provide much faster speeds and more reliable connections.

Q8.Protocols

Protocols are a set of rules that define how data is transmitted and received between devices on a network.

THEORY EXERCISE: What are the differences between HTTP and HTTPS protocols?

HTTP sends data in plain text and is not secure, while **HTTPS** encrypts data using SSL/TLS, making communication secure and safe.

Q9. Application Security

Application security refers to the practices and tools used to protect software applications from threats by preventing, detecting, and fixing security vulnerabilities.

LAB EXERCISE: Identify and explain three common application security vulnerabilities. Suggest possible solutions.

Here are **three common application security vulnerabilities** with **simple explanations and solutions** (lab-friendly):

1. SQL Injection

Explanation: Attackers insert malicious SQL queries to access or modify a database.
Solution: Use prepared statements, input validation, and parameterized queries.

2. Cross-Site Scripting (XSS)

Explanation: Malicious scripts are injected into web pages and executed in users' browsers.
Solution: Validate and sanitize user input, and use proper output encoding.

3. Weak Authentication

Explanation: Poor password practices allow attackers to gain unauthorized access.
Solution: Enforce strong passwords, use multi-factor authentication, and secure login sessions.

THEORY EXERCISE: What is the role of encryption in securing applications?

Encryption protects application data by converting it into an unreadable form, ensuring confidentiality and preventing unauthorized access during storage and transmission.

Q10.Software Applications and Its Types

Software applications are programs designed to perform specific tasks for users.
Types include **system software, application software, and utility software**.

LAB EXERCISE: Identify and classify 5 applications you use daily as either system software or application software.

Application	Type
Windows Operating System	System Software
Google Chrome	Application Software
Microsoft Word	Application Software
WhatsApp	Application Software
VLC Media Player	Application Software

THEORY EXERCISE: What is the difference between system software and application software?

System software controls and manages computer hardware and provides a platform for other software, while **application software** helps users perform specific tasks like writing documents or browsing the internet.

Q11.Software Architecture

Software architecture is the high-level structure of a software system that defines its components, how they interact, and how the system is organized.

LAB EXERCISE: Design a basic three-tier software architecture diagram for a web application.

Here is a **basic three-tier software architecture diagram** for a web application (simple and lab-friendly):

[Presentation Layer]

(Web Browser / UI)

|

v

[Application Layer]

(Web Server / Business Logic)

|

v

[Data Layer]

(Database / Storage)

Layer Roles (short)

Presentation Layer: User interface (browser, frontend)

Application Layer: Processes requests and business logic

Data Layer: Stores and manages data

THEORY EXERCISE: What is the significance of modularity in software architecture?

Modularity makes software easier to develop, test, maintain, and scale by dividing it into independent, manageable components.

Q12. Layers in Software Architecture

Layers in software architecture typically include the **presentation layer, application (business logic) layer, and data layer**, each with a specific role in how the software functions.

LAB EXERCISE: Create a case study on the functionality of the presentation, business logic, and dataaccess layers of a given software system.

Here's a **simple lab-style case study** using an **Online Shopping System** as the example:

Case Study: Three-Layer Architecture in an Online Shopping System

1. Presentation Layer

Functionality:
The presentation layer provides the user interface of the system. In an online shopping system, this includes web pages or mobile app screens where users browse products, add items to the cart, and place orders.

Role:

- Displays product information
- Accepts user input (login details, orders)
- Sends user requests to the business logic layer

---

2. Business Logic Layer

Functionality:
This layer processes user requests and applies business rules. For example, it checks product availability, calculates total price, applies discounts, and validates user credentials.

Role:

- Handles application logic
- Processes orders and payments
- Communicates between presentation and data layers

---

3. Data Access Layer

Functionality:
The data access layer manages interaction with the database. It stores and retrieves data such as user accounts, product details, and order records.

Role:

- Performs database operations (insert, update, delete)
- Ensures secure and efficient data storage
- Provides data to the business logic layer

Conclusion

The separation of presentation, business logic, and data access layers improves **maintainability, security, and scalability** of the software system.

THEORY EXERCISE: Why are layers important in software architecture?

Layers are important in software architecture because they **separate responsibilities**, making software easier to develop, maintain, test, and scale.

Q13.Software Environments

Software environments are the platforms and settings where software is developed, tested, and run, such as **development, testing, and production environments**.

LAB EXERCISE: Explore different types of software environments (development, testing, production).Set up a basic environment in a virtual machine.

Here's a **simple lab-style explanation** with **basic setup steps**:

Types of Software Environments

1. Development Environment

Purpose: Used by developers to write and modify code.
Features: Debug tools, compilers, editors.

---

2. Testing Environment

Purpose: Used to test the software for bugs and errors.
Features: Test data, testing tools, stable configuration.

---

3. Production Environment

Purpose: Live environment used by real users.
Features: High security, optimized performance, real data.

Basic Environment Setup Using a Virtual Machine

Steps:

1. Install a virtualization tool (e.g., VirtualBox or VMware).
2. Create a new virtual machine.
3. Install an operating system (e.g., Linux).
4. Set up required software (compiler, web server, database).
5. Use this VM as a **development or testing environment**.

Conclusion

Using different environments ensures **safe development, proper testing, and stable production deployment**.

THEORY EXERCISE: Explain the importance of a development environment in software production.

A **development environment** is important because it provides a safe space for developers to write, test, and debug code without affecting the live (production) system.

Q14.Source Code

Source code is the human-readable set of instructions written by a programmer that tells a computer how to perform tasks.

THEORY EXERCISE: What is the difference between source code and machine code?

Source code is written by humans in programming languages and is easy to read, while **machine code** is written in binary (0s and 1s) and is directly executed by the computer.

Q15. Github and Introductions

GitHub is an online platform used to store, manage, and share source code using version control (Git). It helps developers collaborate, track changes, and maintain software projects efficiently.

THEORY EXERCISE: Why is version control important in software development?

Version control is important because it **tracks changes, enables collaboration, prevents data loss, and allows developers to revert to earlier versions of code**.

Q16.Student Account in Github

A **GitHub Student Account** provides students with **free access to premium tools and features** through the GitHub Student Developer Pack, helping them learn, collaborate, and build projects efficiently.

THEORY EXERCISE: What are the benefits of using Github for students?

GitHub benefits students by enabling **version control, collaboration, cloud code storage, portfolio building, and access to free learning tools through the Student Developer Pack**.

Q17.Types of Software

The main **types of software** are **system software, application software, and utility software**.

LAB EXERCISE: Create a list of software you use regularly and classify them into the followingcategories: system, application, and utility software.

Here's a **simple lab-style list** classifying commonly used software:

Software	Category
Windows / Linux OS	System Software
Google Chrome	Application Software
Microsoft Word	Application Software
VLC Media Player	Application Software
Antivirus Software	Utility Software

THEORY EXERCISE: What are the differences between open-source and proprietary software?

Open-source software allows users to **view, modify, and share the source code**, while proprietary software has **restricted access** and is owned and controlled by a company.

Q18.GIT and GITHUB Training

Git is a version control system used to track changes in source code, while **GitHub** is an online platform that hosts Git repositories and helps developers collaborate and manage projects.

THEORY EXERCISE: How does GIT improve collaboration in a software development team?

GIT improves collaboration by allowing multiple developers to work on the same project simultaneously, track changes, manage versions, and merge work efficiently without overwriting each other's code.

Q19.Application Software

Application software consists of programs designed to help users perform specific tasks, such as word processing, web browsing, communication, or media playback.

LAB EXERCISE: Write a report on the various types of application software and how they improveproductivity.

Here is a **clear, lab-ready report** you can submit:

LAB EXERCISE: Report on Types of Application Software and Their Role in Improving Productivity

Introduction

Application software refers to programs designed to help users perform specific tasks efficiently. These applications play an important role in improving productivity by automating work, saving time, and improving accuracy.

Types of Application Software

1. Word Processing Software

Examples: Microsoft Word, Google Docs
Productivity Benefits:

- Easy creation and editing of documents
- Spell check and formatting tools save time
- Enables quick sharing and collaboration

---

2. Spreadsheet Software

Examples: Microsoft Excel, Google Sheets
Productivity Benefits:

- Fast calculations and data analysis
- Automatic formulas reduce manual work
- Useful for budgeting, reports, and records

---

3. Presentation Software

Examples: Microsoft PowerPoint, Google Slides
Productivity Benefits:

- Helps create professional presentations quickly
- Visual tools improve communication
- Saves time in explaining ideas

---

4. Communication Software

Examples: Email, Zoom, WhatsApp, Microsoft Teams
Productivity Benefits:

- Enables instant communication
- Supports remote work and collaboration
- Reduces delays in decision-making

---

5. Database and Business Software

Examples: MySQL, ERP systems, CRM software
Productivity Benefits:

- Efficient data storage and retrieval
- Improves accuracy and organization
- Helps manage business operations smoothly

Conclusion

Application software improves productivity by simplifying tasks, reducing manual effort, improving collaboration, and increasing efficiency. Different types of application software are essential in education, business, and everyday life.

THEORY EXERCISE: What is the role of application software in businesses?

Application software helps businesses **automate tasks, manage data, improve communication, and increase efficiency and productivity**.

Q20.Software Development Process

The **software development process** is a series of steps used to design, build, test, and maintain software, typically including **planning, analysis, design, development, testing, deployment, and maintenance**.

LAB EXERCISE: Create a flowchart representing the Software Development Life Cycle (SDLC)

[Requirement Analysis]

|

v

[System Design]

|

v

[Implementation / Coding]

|

v

[Testing]

|

v

[Deployment]

|

v

[Maintenance]

THEORY EXERCISE: What are the main stages of the software development process?

The main stages of the software development process are **requirement analysis, design, implementation, testing, deployment, and maintenance**.

Q21.Software Requirement

Software requirements are detailed descriptions of what a software system should do and the constraints under which it must operate.

LAB EXERCISE: Write a requirementspecification for a simple library managementsystem.

Library Management System – Requirement Specification

1. Introduction

The Library Management System (LMS) is designed to manage library operations such as maintaining book records, member information, and issuing and returning books.

2. Scope

The system will be used by librarians and library members to efficiently manage books, track borrow/return activities, and reduce manual work.

3. Functional Requirements

- The system shall allow librarians to add, update, and delete book records.
- The system shall allow users to search books by title, author, or ISBN.
- The system shall allow members to borrow and return books.
- The system shall track due dates and availability status of books.
- The system shall allow librarians to manage member records.

4. Non-Functional Requirements

- The system shall be easy to use and user-friendly.
- The system shall respond to user actions within acceptable time limits.
- The system shall ensure data security and user authentication.
- The system shall be reliable and available during library working hours.

5. Constraints

- The system shall run on standard computers.
- The system shall use a centralized database.

6. Conclusion

This requirement specification defines the basic functional and non-functional requirements of a simple Library Management System.

THEORY EXERCISE: Why is the requirement analysis phase critical in software development?

The **requirement analysis phase** is critical because it clearly defines what the software must do, helps avoid errors and misunderstandings, and ensures the final system meets user needs.

Q22.Software Analysis

Software analysis is the process of understanding and evaluating software requirements, structure, and behavior to ensure it meets user needs and works efficiently.

LAB EXERCISE: Perform a functional analysis for an online shopping system.

Functional Analysis – Online Shopping System

Main Functions:

1. **User Management** – User registration, login, logout, profile management
2. **Product Management** – Browse products, view details, search, filter, and categorize items
3. **Shopping Cart** – Add items, update quantities, remove items, view cart
4. **Order Processing** – Place orders, confirm orders, generate order IDs
5. **Payment Processing** – Select payment method, process payments, confirm payment status

6. **Order Tracking** – View order history, track order status
7. **Inventory Management** – Update stock levels after purchase
8. **Admin Functions** – Manage products, prices, users, and orders

Inputs: User details, product selections, payment information
Outputs: Order confirmation, payment receipt, order status updates.

THEORY EXERCISE: What is the role of software analysis in the development process?

The role of software analysis in the development process is to identify and define user requirements, understand system needs, and assess feasibility. It helps ensure the software is correctly designed, reduces errors and risks, and provides a clear foundation for design, implementation, and testing.

Q23.System Design

System design is the process of defining a system's architecture, components, modules, data flow, and interfaces to meet specified requirements efficiently and reliably.

LAB EXERCISE: Design a basic system architecture for a food delivery app

Basic System Architecture – Food Delivery App

1. Client Layer

- Mobile App (Customer)
- Mobile App / Web Panel (Restaurant)
- Admin Dashboard

2. Application Layer (Backend)

- User Service (login, profiles)
- Restaurant & Menu Service
- Order Management Service
- Payment Service
- Delivery & Tracking Service
- Notification Service (SMS/Push)

3. Data Layer

- User Database
- Restaurant & Menu Database
- Orders Database
- Payments Database

4. External Services

- Payment Gateway
- Maps & GPS Service
- Notification APIs

Flow:
User places order → Backend processes order → Payment handled → Restaurant prepares food → Delivery tracked → User notified ✅.

THEORY EXERCISE: What are the key elements of system design?

The key elements of system design include system architecture, components and modules, data design, interfaces, and data flow. It also involves defining hardware and software requirements, security, performance, and scalability considerations.

Q24.Software Testing

Software testing is the process of evaluating software to find defects and verify that it works correctly, meets requirements, and delivers expected results.

THEORY EXERCISE: Why is software testing important?

Software testing is important because it helps identify defects, ensures the software meets requirements, improves quality and reliability, reduces development risks, and prevents costly failures after deployment.

Q25.Maintenance

Software maintenance is the process of modifying and updating software after deployment to fix bugs, improve performance, adapt to changes, and add new features.

LAB EXERCISE: Document a real-world case where a software application required criticalmaintenance.

Real-World Case: Critical Software Maintenance in the Healthcare System

A hospital management system required critical maintenance after frequent system crashes during patient record access. The issue was caused by increased data volume and outdated database queries, leading to slow performance and downtime.

Maintenance Performed:

- Optimized database queries
- Upgraded server infrastructure
- Fixed software bugs causing memory leaks
- Improved backup and recovery mechanisms

Outcome:
System performance improved significantly, downtime was reduced, and patient data access became reliable and secure.

THEORY EXERCISE: What types of software maintenance are there?

There are four main types of software maintenance:

1. **Corrective maintenance** – Fixes bugs and errors found after deployment.
2. **Adaptive maintenance** – Modifies software to work in a changed environment (new OS, hardware, regulations).
3. **Perfective maintenance** – Enhances performance, usability, or adds new features.
4. **Preventive maintenance** – Improves code and structure to reduce future problems.

Q26.Development

Software development is the process of designing, building, testing, and maintaining software applications to meet user and business requirements.

THEORY EXERCISE: What are the key differences between web and desktop applications?

The key differences between web and desktop applications are:

- **Platform dependency:** Web applications run in a browser on any device, while desktop applications are installed on a specific operating system.
- **Installation:** Web apps do not require installation; desktop apps must be installed locally.
- **Updates:** Web applications are updated centrally on the server, while desktop apps require user-side updates.
- **Accessibility:** Web apps can be accessed from anywhere with an internet connection; desktop apps usually run offline.
- **Performance:** Desktop applications generally have better performance and access to system resources compared to web applications.

Q27. Web Application

A **web application** is a software application that runs on a web server and is accessed through a web browser over the internet, without needing installation on the user's device.

THEORY EXERCISE: What are the advantages of using web applications over desktop applications?

The advantages of using web applications over desktop applications include:

- **Platform independence** – run on any device with a web browser
- **No installation required** – easy access for users
- **Centralized updates** – changes are applied instantly for all users
- **Accessibility** – available anytime, anywhere with internet access
- **Lower maintenance cost** – easier deployment and support
- **Scalability** – can easily support a large number of users
-
-

Q28. Designing

Designing is the process of planning and creating the structure, layout, and functionality of a system or application to meet specified requirements before development begins.

THEORY EXERCISE: What role does UI/UX design play in application development?

UI/UX design plays a crucial role in application development by ensuring the application is easy to use, visually appealing, and provides a positive user experience. It helps users interact efficiently with the system, increases user satisfaction, and improves usability and adoption of the application.

Q29. Mobile Application

A **mobile application** is a software program designed to run on smartphones or tablets, providing specific services or functions through a mobile operating system such as Android or iOS.

THEORY EXERCISE: What are the differences between native and hybrid mobile apps?

The differences between native and hybrid mobile apps are:

- **Development:** Native apps are built using platform-specific languages (Swift/Objective-C for iOS, Kotlin/Java for Android), while hybrid apps use web technologies like HTML, CSS, and JavaScript.
- **Performance:** Native apps offer better performance and smoother user experience; hybrid apps may be slower.
- **Access to device features:** Native apps have full access to device hardware and APIs, while hybrid apps have limited access through plugins.
- **Cost and time:** Native apps are more expensive and time-consuming to develop; hybrid apps are faster and cheaper.
- **Code reusability:** Hybrid apps allow a single codebase for multiple platforms, while native apps require separate codebases.
-

Q30. DFD (Data Flow Diagram)

A **DFD (Data Flow Diagram)** is a graphical representation that shows how data moves through a system, illustrating data inputs, processes, data storage, and outputs.

LAB EXERCISE: Create a DFD for a hospital managementsystem.

DFD – Hospital Management System (Level 0 / Context Diagram)

External Entities:

- Patient
- Doctor
- Admin
- Laboratory

Main Process:

- Hospital Management System

Data Flows:

- Patient → System: Registration details, appointment requests
- System → Patient: Appointment confirmation, bills, reports
- Doctor → System: Diagnosis, prescriptions
- System → Doctor: Patient records, schedules
- Laboratory → System: Test results
- System → Laboratory: Test requests
- Admin ↔ System: Staff details, billing, reports

This DFD shows the overall flow of data between users and the hospital management system.

THEORY EXERCISE: What is the significance of DFDs in system analysis?

DFDs are significant in system analysis because they visually represent how data flows within a system. They help analysts understand system processes, identify data sources and destinations, improve communication among stakeholders, and detect inefficiencies or missing requirements early in the development process.

Q31. Desktop Application

A **desktop application** is a software program installed and run on a personal computer or laptop, designed to work on a specific operating system such as Windows, macOS, or Linux.

THEORY EXERCISE: What are the pros and cons of desktop applications compared to webapplications?

Pros of Desktop Applications:

- Better performance and faster response

- Full access to system resources and hardware
- Can work offline
- More secure for sensitive local data

Cons of Desktop Applications:

- Platform dependent (OS-specific)
- Requires installation and manual updates
- Limited accessibility (device-specific)
- Higher maintenance and distribution cost compared to web applications.
-

Q32. Flow Chart

A **flowchart** is a diagrammatic representation of a process or algorithm, showing step-by-step flow of control using standardized symbols such as ovals, rectangles, diamonds, and arrows.

LAB EXERCISE: Draw a flowchart representing the logic of a basic online registration system.

Flowchart – Basic Online Registration System (Logic)

Start
↓
User opens registration page
↓
User enters details (name, email, password)
↓
Are all required fields filled?

- **No** → Show error message → Return to input
- **Yes** → Continue
  ↓
  Validate email and password format
  ↓
  Is data valid?
- **No** → Show validation error → Return to input
- **Yes** → Continue
  ↓
  Check if user already exists
  ↓
  User exists?

- **Yes** → Show “User already registered” message
- **No** → Save user details in database
  ↓
  Display registration success message
  ↓
  End

(This textual flow represents the same logic as a drawn flowchart using standard symbols.)

THEORY EXERCISE: How do flowcharts help in programming and system design?

Flowcharts help in programming and system design by visually representing the logic and flow of a process. They make complex systems easier to understand, help identify errors or inefficiencies early, improve communication among developers and stakeholders, and serve as a clear guide during coding and documentation.

-

: